How Much of Reported Quantization Degradation Is Measurement? Decomposing Accuracy Loss into Competence, Compliance, and Truncation

Anonymous ACL submission

Abstract

[**TODO:** Abstract — written in W5 (C7), after results freeze.]

1 Introduction

2 Related Work

A live contradiction about generation length. Whether low-bit quantization lengthens chain-of-thought output is currently disputed. Lian et al. (2026) report that INT4/INT3 quantization can preserve accuracy while *increasing* reasoning-token usage, and propose a CoT Token Inflation Ratio to quantify it; Lotfi et al. (2026) likewise find that aggressive post-training quantization lengthens CoT. Liu et al. (2025) report the opposite: "contrary to expectations, quantized models do not exhibit increased output lengths." We take no position on which generator behaviour is correct, because the disagreement has a consequence that neither side measures. Under a *fixed* generation cap, a model that emits longer traces is scored wrong more often whenever its answer falls beyond the cap — a scoring outcome, not a competence one. No study on either side of this dispute sweeps the generation cap, so no study can separate the two. That separation is our H1.

Degradation has already been partly explained — by other causes. Kurtic et al. (2025) show that reported quantization degradation was overstated, and attribute the excess to untuned quantization-side hyperparameters. We treat that result as established and *prior*, not as a target: our claim concerns the residual that remains after quantization-side tuning, and our mechanism is orthogonal to theirs. Lotfi et al. (2026) supply the most direct antecedent for the present work, finding that in as many as 52% of quantized-model failures the model reaches the correct answer in an intermediate step but does not emit it as the final answer. Their interest is in fixing the generator, via a logit penalty on overthinking markers; ours is in what such failures do to a reported accuracy delta when they are left in place.

The centre of gravity runs the other way. The dominant finding in this literature is that benchmarks *understate* quantization damage, not overstate it. Marchisio et al. (2024) find a 1.7% automatic drop in Japanese corresponds to 16.0% by human judgement, and Dutta et al. (2024) show that models matched on accuracy still diverge behaviourally — the "flips" framing, which is theirs and which we do not claim. We regard both directions as simultaneously true: an accuracy delta can over-penalise formatting *and* under-detect behavioural change, because these are different quantities. Our contribution is to the first, and it does not license any conclusion about the second.

Calibrating extraction, not loosening it. An argument that evaluation should simply be more permissive is already refuted. Ren et al. (2026) find that 8.2–44.1% of answers graded correct are reached by shortcut hacking, so lenient extraction over-credits as surely as strict extraction under-credits. We therefore treat the strict/permissive contrast as a measurement instrument to be characterised rather than a knob to be turned in one direction, and we report both rungs throughout.

Prompt- and protocol-sensitivity. Hua et al. (2025) ask whether prompt sensitivity is a genuine model property or an artifact of evaluation, and we are structurally the sequel: they vary the prompt, we vary the scorer, and both ask whether a reported effect survives the instrument. Their null result on MATH deserves emphasis rather than a footnote — extraction there already uses symbolic equivalence, so a well-engineered verifier leaves little for a protocol effect to explain. This

bounds our claim rather than supporting it, and it is why our task selection targets benchmarks whose canonical extractors are regular expressions.

The nearest prior work, and what it deliberately designed out. Pape et al. (2026) is the closest study to our §4.1 and must be distinguished precisely. They survey 35,000 publications, find that only 36% of papers without released code name even their inference backend, and show that switching backend alone moves GSM8K by 16.6 points. We adopt their framing and their survey rather than repeating it. But their controlled comparison runs at *batch size one*, chosen explicitly to prevent batching-induced instability, holds precision at FP16, and varies only random seeds within an engine; batch invariance and quantized kernel selection do not enter. The instability they design out is the one we make the object of study: we hold the engine, the weights and the decoding configuration fixed, vary only which items share a batch, and attribute the resulting divergence to a named quantized kernel by intervention. Their root-cause analysis independently supports our wider point — their two largest outliers were a spurious BOS injection and an unnoticed repetition penalty, worth 8.3 and 11.7 points, both configuration defaults rather than model properties.

Sclar et al. (2024) establish that spurious formatting choices move scores substantially, and Jo et al. (2025) and Yu et al. (2025) show that answer extraction from reasoning traces is itself an unreliable step. Schaeffer et al. (2023) provide the methodological template we follow: a headline effect reattributed, in part, to the measurement instrument. The lm-evaluation-harness (Biderman et al., 2024) is our instrument, and its canonical filters are the object of study rather than an implementation detail.

Serving configuration is a confound we pin, not an axis we explore. Pape et al. (2026) find that inference-backend choice alone moves benchmark scores by up to 16.6 points, and Masoudian et al. (2026) attribute roughly 39% of out-of-the-box variance to it. We read this as a reason to freeze the engine, version, and every decoding parameter across all arms, which we do and report; a floating serving configuration would make precision effects unidentifiable. We claim no novelty on this axis.

Adjacent evaluations. Broad compression benchmarks (von Rad et al., 2026; Wang et al., 2025) and task-difficulty studies (Lee et al., 2025) establish the accuracy landscape we work inside; Lee et al. (2025) in particular report that quantization disrupts instruction-following, which is the strongest available objection to our framing and which our compliance/competence split is designed to address directly rather than deflect. Prasad and Pal (2026) run a factorial quantization analysis, so factorial design is not our contribution; their outcome domain is safety alignment under sampling temperature, which is disjoint from ours. Kurt (2026) report the observation this paper sets out to explain — strict-versus-flexible GSM8K gaps of 41 to 64 points across fourteen `llama.cpp` quantizations — without interpreting the gap; we reproduce their numbers as an external anchor. Oladri et al. (2026) and Rasmussen and Qin (2026) report related failure-mode and scoring-rule effects, the former over FP32/FP16/NF4 only and so without bearing on GPTQ or AWQ.

What is new here. Permissive extraction is well known to raise absolute scores. What has not been established is whether it changes the quantization *difference*: an artifact in a level need not be an artifact in a delta, and the two are routinely conflated. We decompose the measured strict delta into competence, compliance, and truncation components, pre-register the decomposition before any generation exists, and report the residual precision effect that survives it.

3 Method

3.1 Experimental design

We evaluate three instruction-tuned models spanning a 3–8B size gradient: Qwen2.5-3B-Instruct, Phi-3.5-mini-instruct, and Llama-3.1-8B-Instruct. Each is pinned by commit SHA rather than tag, since tags move and reproducibility holds only within identical versions.

Five precision arms form a dose–response rather than a two-point comparison: `bf16`, GPTQ-8bit, GPTQ-4bit, AWQ-4bit, and bitsandbytes NF4. A monotone bf16 $\rightarrow$ 8-bit $\rightarrow$ 4-bit trend is a substantially stronger internal-validity argument than a single contrast, because a protocol artifact has no reason to track bit-width monotonically.

All manipulations are one-factor-at-a-time from a single fixed reference condition, so every re-

ported effect is attributable to the one factor varied. NF4 is included in the reference condition but excluded from the token-budget sweep, and we state the reason rather than the exclusion alone: NF4 has no CUDA kernels in our serving engine, and it is quantized *in flight* at load rather than loaded from a pre-made checkpoint. That is a backend difference layered on the bit-width difference, which we name explicitly rather than absorb.

3.2 Quantization and calibration

This is the most likely attack surface on the paper, so we state it in full. Both GPTQ and AWQ are produced by one tool, through one calibration code path, at *one* normalised budget: 512 sequences of at most 2048 tokens drawn from a single pinned shuffle of `ultrachat_200k`, identical for every arm, every bit-width, and every model. We report the realised token count rather than the product of those two numbers, since sequences shorter than the cap contribute their own length.

Normalisation is necessary because the reference implementation's own examples differ by **8**× in calibration budget between the two methods (GPTQ at 512×2048 symmetric; AWQ at 256×512 asymmetric). Accepting those defaults would mean a GPTQ-versus-AWQ difference confounded with an 8× calibration difference *and* a symmetry difference. We therefore normalise AWQ to symmetric 4-bit weight-only quantization, which is a deliberate deviation from its published example, and we flag it as such.

Group size is 128 and activation reordering is disabled for every GPTQ arm, matching the vendor's own released checkpoints. Disabling reordering is set explicitly rather than left to the library, whose default silently applies a static reordering. Every parameter above is hashed into a configuration digest carried on each generation record, so a reader can verify that no arm drifted.

All arms are self-quantized with identical settings. Vendor checkpoints are used only as an external cross-check, never mixed into the main matrix, because a widely used third-party GPTQ checkpoint for one of our models enables activation reordering against the vendor's own setting — exactly the kind of silent confound this design exists to exclude.

3.3 The scoring ladder

Every generation is stored once and scored many times. We apply a ladder of extractors to the *same* stored text, so the entire ladder costs zero additional generation and every rung sees byte-identical inputs:

1. **strict** — the evaluation harness's canonical regular expression, used verbatim.
2. **strict-last** — the same expression selecting the *last* match instead of the first, which separates answer *position* from extraction *strictness*.
3. **permissive** — the harness's flexible extractor, with its rules enumerated in the pre-registration.
4. **symbolic** — equivalence checking rather than string matching.
5. **judge** — a trained answer verifier (Chen et al., 2025).

The headline contrast is rung 1 versus rung 2. Both are deterministic, auditable, and judge-independent, which is what makes judge validation a robustness check rather than a load-bearing dependency: if the judge fails its agreement gate, the paper's main claim is unaffected. Rung 1b exists because a harness that takes the first match will score a model that restates its answer differently from one that does not, independent of correctness.

3.4 Compliance, competence, and truncation

We decompose the measured strict delta into three additive components: a *competence* component (the model does not know the answer), a *compliance* component (it knows but does not emit the answer in the expected form), and a *truncation* component (it would have emitted the answer but ran out of budget). Recovering the third requires two fields that cannot be reconstructed after the fact — the engine's termination reason and the generated token count — so both are recorded on every generation, and our writer refuses any record missing either.

Compliance is treated as a dependent variable in its own right, not merely as a correction term. We also distinguish an answer being *extractable* from the model having *committed* to it: a fallback rule that returns the last number in a truncated trace has extracted something the model never asserted, and counting that as compliance would inflate the very component we are trying to measure.

3.5 Serving configuration and determinism

The serving configuration is frozen and identical across every arm: one engine, one pinned version, one set of decoding parameters, greedy decoding throughout. This is not incidental rigour. Backend choice alone moves benchmark scores by up to 16.6 points (Pape et al., 2026) and accounts for roughly 39% of out-of-the-box variance (Masoudian et al., 2026), so a serving configuration that floated across arms would make precision effects unidentifiable regardless of sample size.

Greedy decoding is not by itself sufficient for reproducibility, because inference engines batch requests and batched reductions are not order-invariant. We therefore enable batch-invariant kernels, run with tensor parallelism of one (batch invariance is broken above one), disable prefix caching (it changes greedy output and is explicitly outside the invariance guarantee, and it would otherwise help throughput considerably on our shared few-shot prefixes), and fix the maximum number of co-resident sequences to a value *measured* rather than assumed: the largest setting that is bitwise reproducible across repeated identical runs for *every* precision arm, which is a minimum over arms because a single shared value is required.

We report that value, the per-arm bitwise disagreement rates behind it, the GPU model, and the CUDA and driver versions. Where a quantized arm is less reproducible than `bf16`, the difference is a noise floor on every contrast involving that arm, and we pre-commit to claiming no effect smaller than it.

4 Results

4.1 Evaluation-set ordering (H8)

We serve every arm through a single engine version (1.0.0) with batch-invariant kernels enabled, tensor parallelism disabled, prefix caching off, and a fixed decoding configuration, then vary one parameter that no evaluation protocol we are aware of reports: *the order in which items are submitted*. Because requests are batched dynamically, an item's position determines which other items share its batch, and therefore the shapes over which the engine's reductions are performed.

Holding the ordering fixed, generation is exactly reproducible: the repetition floor is 0.000 % in all 15 cells. So every difference reported below is attributable to ordering alone, not to run-to-run variation.

	strict		permissive	
arm	SD	range	SD	range
BF16	0.00	0.00	0.00	0.00
GPTQ W4A16	1.00	3.20	0.67	2.40
AWQ W4A16	0.80	2.60	1.14	3.80
bnb NF4	0.06	0.20	0.06	0.20

Table 1: Accuracy across 10 orderings of the same 500 items, in accuracy points, at $B = 64$. The standard deviation is the inferential quantity; the range is descriptive only. `bf16` is identical under every ordering.

Aggregate accuracy moves, and only for quantized arms. Table 1 gives the standard deviation of accuracy across 10 orderings of the same 500 items at the served batch size we adopt. For `bf16` the accuracy is *identical* under all 10 orderings, at both scoring rungs: 2 of 2 cells show exactly zero variation. Every quantized arm moves, by up to 1.14 points of standard deviation.

Why we report a standard deviation and not a spread. A minimum–maximum spread is monotone in the number of orderings sampled, so it grows with effort rather than converging. We can show this rather than assert it: the same quantity measured over three orderings gave a spread of 1.6 points and over 10 gives 3.8, with no change to the underlying effect. Any spread reported over a handful of runs is therefore a lower bound on the spread its own protocol would have produced with more. The standard deviation, by contrast, stayed within a narrow band across arms and rungs.

Three properties matter more than the magnitude. The shift is **deterministic** given an ordering rather than sampling noise, so it cannot be averaged away by repeated runs. It is **bidirectional** — accuracy moves both up and down across orderings, so no ordering is the correct one and the effect cannot be calibrated away. And it is **attributable by intervention** rather than inferred, as follows.

The mechanism is a named kernel. Disabling Marlin and Machete through `VLLM_DISABLED_KERNELS`, changing nothing else, drives order-induced divergence to zero on 2 arms (Table 2) — on generated text *and* on accuracy. These are the quantized matrix-multiply kernels vLLM selects by default, and they are not covered by its batch-invariant mode: the AWQ kernel path carries a batch-invariance guard, the GPTQ path carries none, and the sampler and

	text divergence		accuracy SD	
arm	Marlin on	off	Marlin on	off
GPTQ W4A16	33.4	0.0	0.12	0.00
AWQ W4A16	43.6	0.0	0.70	0.00
bnb NF4	31.4	31.4	0.23	0.23

Table 2: Paired intervention at $B = 8$: identical inputs, identical engine, with and without the Marlin and Machete kernels. Text divergence and accuracy SD in percentage points. bnb NF4 is unaffected, indicating a second and distinct mechanism.

kernel autotuning paths are likewise unguarded. We report this as a defect in the feature rather than in either kernel; AWQ has the guard and is nonetheless affected, so the guard as implemented is insufficient.

Two controls rule out the obvious alternatives. Enabling kernel autotuning leaves every figure bit-identical, so the effect is not an autotuning artifact. And bnb NF4 is unmoved by the intervention while remaining order-sensitive, so at least one further mechanism exists that is Marlin-independent; we note that it becomes *less* order-sensitive at larger batch sizes, the reverse of every other arm, and leave the cause unresolved.[1]

The frontier. Table 3 reports divergence at all three measurement levels and all three batch sizes, rather than only at the configuration we adopt. The three levels differ by roughly an order of magnitude: generated text diverges far more often than the extracted answer, which diverges more often than scored correctness. Reporting text divergence alone would substantially overstate the impact on any reported number, and reporting correctness alone would conceal how much of the trace is unstable.

Notably the permissive rung is the more order-sensitive of the two at the answer level, because it accepts the last number anywhere in the trace and the trace is what diverges, whereas strict extraction requires a committed marker that is comparatively stable. Leniency therefore buys recovery of truncated answers at the cost of exposure to batch-composition noise — which is an argument for calibrating extraction rather than loosening it

[1] `bitsandbytes` is proposed for out-of-tree maintenance in vLLM RFC #39583, on performance grounds. Two explanations are consistent with our data — a data-dependent reduction whose result concentrates at larger batch sizes, or a different kernel-selection path entirely — and we do not have evidence to separate them.

	$B = 8$			$B = 32$			$B = 64$	
arm	text	str.	perm.	text	str.	perm.	text	str.
BF16	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
GPTQ W8A16	29.8	3.6	4.6	31.8	4.2	5.0	29.8	4.0
GPTQ W4A16	33.4	4.2	3.0	40.0	5.4	3.6	39.6	4.8
AWQ W4A16	43.6	4.0	6.4	45.6	4.2	7.0	47.0	3.8
bnb NF4	31.4	4.8	4.0	2.6	0.2	0.2	1.4	0.2

Table 3: Order-induced divergence at three measurement levels — generated text, and correctness under strict and permissive extraction — by served batch size B. The frontier, not the single configuration we adopt, is the result: it bounds what a third party can reproduce.

(§3.3).

Scope. [**TODO:** H8 scope — restate once the 10-ordering confirmatory run (pre-registered, Appendix C) lands: SD over 10 orderings, 4 arms at $B = 64$, with the CORRECTED terminators, so it is not directly comparable to the 3-ordering figures above and supersedes rather than extends them. Current SD is weakly estimated at 3 orderings. Also add the one comparative sentence — ordering spread as a fraction of the bit-width effect measured on the SAME items — which requires the reference runs and MUST NOT be sourced from published deltas on other models and tasks, which §2 argues are mutually incomparable.]

4.2 Truncation (H1)

[**TODO:** H1 — degradation vs generation budget; mean/median n_gen_tokens by precision with CIs, settling 2606.25519 vs 2504.04823.]

4.3 Recoverable failures (H2)

[**TODO:** H2 — gold-in-trace rate, permissive and conservative variants, always reported net of the incidental-match false-positive floor.]

4.4 Precision × scoring interaction (H3)

[**TODO:** H3 — GLMM interaction as an odds ratio and as a marginal effect in accuracy points with 95% CI, plus paired McNemar. A NULL IS PUBLISHABLE and must not read as failure.]

4.5 Decomposition (H4)

[**TODO:** H4 — THE CENTREPIECE TABLE. Design it first and let it drive what gets logged. Always-compliant stratum size as a first-class number; compliance-transition matrix; Manski bounds.]

4.6 Prompt format (H5) and the structural control (H6)

[**TODO:** H5 zero-shot arm; H6 ARC-Challenge under log-likelihood vs generative scoring on the same 500 items — the gap between modes IS the extraction cost.]

4.7 Statistical resolution of published deltas (H7)

[**TODO:** H7 — scrupulously neutral: "many papers report aggregate deltas without a paired test", NEVER "the deltas are noise".]

5 Discussion

6 Conclusion

Limitations

Ethics Statement

Acknowledgments

References

Stella Biderman, Hailey Schoelkopf, Lintang Sutawika, Leo Gao, Jonathan Tow, Baber Abbasi, Alham Fikri Aji, Pawan Sasanka Ammanamanchi, Sidney Black, Jordan Clive, Anthony DiPofi, Julen Etxaniz, Benjamin Fattori, Jessica Zosa Forde, Charles Foster, Jeffrey Hsu, Mimansa Jaiswal, Wilson Y. Lee, Haonan Li, and 11 others. 2024. Lessons from the trenches on reproducible evaluation of language models. *arXiv preprint arXiv:2405.14782*.

Ding Chen, Qingchen Yu, Pengyuan Wang, Mengting Hu, Wentao Zhang, Zhengren Wang, Bo Tang, Feiyu Xiong, Xinchi Li, Chao Wang, Minchuan Yang, and Zhiyu Li. 2025. xverify: Efficient answer verifier for reasoning model evaluations. *arXiv preprint arXiv:2504.10481*.

Abhinav Dutta, Sanjeev Krishnan, Nipun Kwatra, and Ramachandran Ramjee. 2024. Accuracy is not all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Andong Hua, Kenan Tang, Chenhe Gu, Jindong Gu, Eric Wong, and Yao Qin. 2025. Flaw or artifact? rethinking prompt sensitivity in evaluating llms. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Hwiyeol Jo, Joosung Lee, Jaehone Lee, Sang-Woo Lee, Joonsuk Park, and Kang Min Yoo. 2025. Finding answers in thought matters: Revisiting evaluation on large language models with reasoning. *arXiv preprint arXiv:2510.14773*.

Uygar Kurt. 2026. Which quantization should i use? a unified evaluation of llama.cpp quantization on llama-3.1-8b-instruct. *arXiv preprint arXiv:2601.14277*.

Eldar Kurtic, Alexandre Marques, Shubhra Pandit, Mark Kurtz, and Dan Alistarh. 2025. "give me bf16 or give me death"? accuracy-performance trade-offs in llm quantization. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*.

Jemin Lee, Sihyeong Park, Jinse Kwon, Jihun Oh, and Yongin Kwon. 2025. Exploring the trade-offs: Quantization methods, task difficulty, and model size in large language models from edge to giant. In *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence (IJCAI)*.

Xinyu Lian, Walid Krichene, Beichen Huang, Masahiro Tanaka, Olatunji Ruwase, Li Zhang, and Minjia Zhang. 2026. Quantization inflates reasoning: Token inflation as a hidden cost of low-bit reasoning models. *arXiv preprint arXiv:2606.25519*.

Ruikang Liu, Yuxuan Sun, Manyi Zhang, Haoli Bai, Xianzhi Yu, Tiezheng Yu, Chun Yuan, and Lu Hou. 2025. Quantization hurts reasoning? an empirical study on quantized reasoning models. In *Conference on Language Modeling (COLM)*.

Sanae Lotfi, Polina Kirichenko, Steven Li, and Zechun Liu. 2026. Quantized reasoning models think they need to think longer, but they do not. *arXiv preprint arXiv:2606.00206*.

Kelly Marchisio, Saurabh Dash, Hongyu Chen, Dennis Aumiller, Ahmet Üstün, Sara Hooker, and Sebastian Ruder. 2024. How does quantization affect multilingual llms? In *Findings of the Association for Computational Linguistics: EMNLP 2024*.

Shahed Masoudian, Passant Shafaei, Monorama Swain, and Markus Schedl. 2026. What we observe as llm behavior can be a side-effect of inference backend. *arXiv preprint arXiv:2608.04714*.

Renuka Oladri, Mohan Vamsi Varadaraju Priya, and Jerry Wu. 2026. Silent failures in quantized llm reasoning: A taxonomy-based analysis of hollow convergence and failure mode shifts. *arXiv preprint arXiv:2607.09999*.

David Pape, Jonathan Evertz, and Lea Schönherr. 2026. The silent hyperparameter: Quantifying the impact of inference backends on llm reproducibility. *arXiv preprint arXiv:2605.19537*.

Hari Prasad and Ritam Pal. 2026. The joint effect of quantization and sampling temperature on llm safety alignment: A factorial analysis. *arXiv preprint arXiv:2606.29581*.

Anton Rasmussen and Hong Qin. 2026. When calibration depends on the scoring rule: Quantized biomedical llm classification. *arXiv preprint arXiv:2608.03854*.

Xuan Ren, Weiqi Zhai, Tianle Pu, Yihua Zhu, Hu Wei, and Bing Zhao. 2026. Right answer, wrong method: Shortcut hacking misleads the evaluation of llm reasoning on frontier science benchmarks. *arXiv preprint arXiv:2608.02442*. Work in progress.

Rylan Schaeffer, Brando Miranda, and Sanmi Koyejo. 2023. Are emergent abilities of large language models a mirage? In *Advances in Neural Information Processing Systems (NeurIPS)*.

Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. 2024. Quantifying language models' sensitivity to spurious features in prompt design or: How i learned to start worrying about prompt formatting. In *International Conference on Learning Representations (ICLR)*.

Jonathan von Rad, Yong Cao, and Andreas Geiger. 2026. Unicomp: A unified evaluation of large language model compression via pruning, quantization and distillation. *arXiv preprint arXiv:2602.09130*.

Jiacheng Wang, Yejun Zeng, Jinyang Guo, Yuqing Ma, Aishan Liu, and Xianglong Liu. 2025. Slmquant: Benchmarking small language model quantization for practical deployment. *arXiv preprint arXiv:2511.13023*.

Qingchen Yu, Zifan Zheng, Shichao Song, Zhiyu Li, Feiyu Xiong, Bo Tang, and Ding Chen. 2025. xfinder: Large language models as automated evaluators for reliable evaluation. In *International Conference on Learning Representations (ICLR)*.

A Frozen configuration

B Annotation rubric

C Pre-registration